EECS 485 Web Systems Introduction

Prof. Almomani

Announcements

- Please read the syllabus and eecs485.org
- Lab starts this week.
- Project 1 due soon.

About Almomani

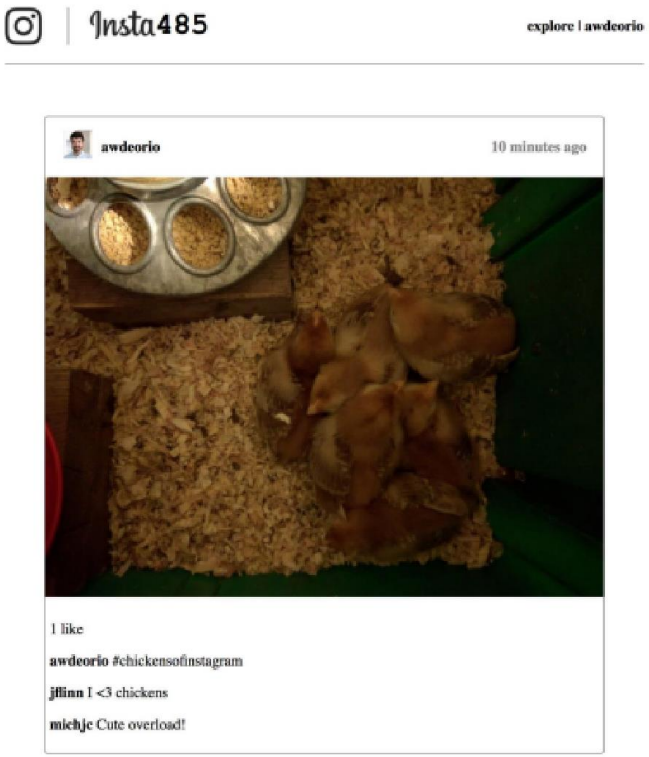
- Prof. Almomani.
- Ph.D. in CS (Computer Vision).
- Taught EECS 484 & 485 multiple times.

3 Big Questions of EECS 485

- How do websites work?
- How do search engines work?
- How can we build website to handle millions/billions of users?

Some things you'll build

- Instagram
  - 3 different ways
  - Projects 1-3
- MapReduce
  - Project 4
- Search engine
  - Project 5



Some technologies you'll use

- HTML/CSS
- Shell scripting
- SQL
  - SQLite database
- Python
  - Flask web framework
- JavaScript
  - React/JS framework
- Hadoop.

Agenda

- What is EECS 485 about?
- **Logistics**
- How the web works
- Python

Course format

- Lectures
  - In-person, recorded
  - No attendance required
- Labs
  - In-person
  - One lab will be recorded
  - No attendance required
  - Attend any section at any time
- Office hours
  - In-person and Remote
  - See calendar on eecs485.org

Getting help

- eecs485.org
  - Links to all course resources
- edstem.org
  - Best for technical questions and project updates
- eecs485staff@umich.edu
  - Best for non-technical questions
- almomani@umich.edu (individual professor email addresses)
  - Best for confidential matters

Projects and Collaboration

Task	Collaboration
Project 1	No
Projects 2-5	Groups of 2 - 3

- You can modify your group between projects
- For those retaking the course: you may work in a group, or you may reuse your old code, but not both.
- We can help you find a group (optional). Watch for email.

Lab

- **YES** we have lab this week

- If you're on the waitlist, attend any lab, or watch the recording.
  - Materials are posted on eecs485.org

- Expect first segment project intro and lecture review
- Second segment hands-on help with projects

Exams

- Put the dates in your calendar
  - See eecs485.org
  - No make-up exam
- In-person, paper
- Cheat sheet OK
- Format
  - Part multiple choice
  - Part code and free response
- Content
  - Concepts from lecture
  - Code from projects

Grades

Projects (5 x 10%)	50%
Midterm	25%
Final	25%

- Draft letter grade after the midterm

Agenda

- What is EECS 485 about?
- Logistics
- **How the web works**
- Python

How the web works

- You click a link on a website. What do you think happens next?
  A. Your computer asks another computer for information
  B. The page is already saved on your computer
  C. A picture of the page is sent to you
  D. Something else?

How the web works

1. The **client** asks the **server** for a page
2. The **server** sends files back
3. The **client's** browser displays the page

- This is called **request response cycle**.

client

internet

server

How the web works

Client: You

Server: Amazon.com

internet

How the web works

Client: You

Request: What's at amazon.com?

internet

Server: amazon.com

How the web works

Client: You

Request: What's at amazon.com?

internet

Response: Files (HTML, CSS)

Server: amazon.com

How the web works

Client: You

Client's browser renders the Files

Request: What's at amazon.com?

internet

Response: Files (HTML, CSS)

Server: amazon.com

What does a server respond with?

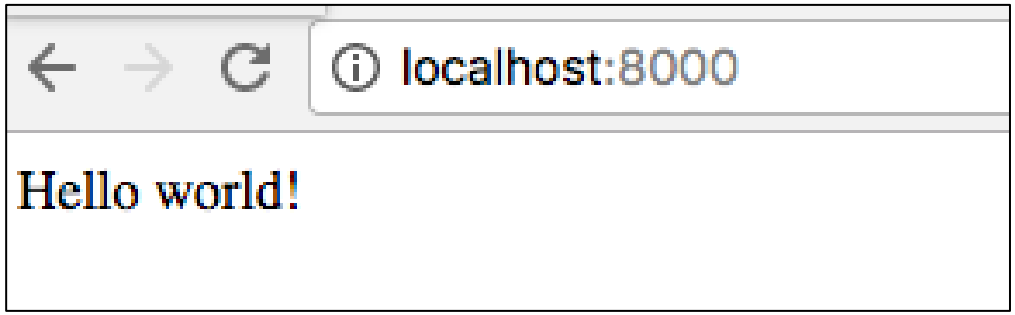
- A server might respond with different kinds of files.
  - HTML
  - CSS
  - JavaScript

- We'll talk about JavaScript later in the semester

HTML: HyperText Markup Language

- HTML describes the **content** on a page
- Example `index.html`

```
<!DOCTYPE html>
<html lang="en">
  <body>
    Hello world!
  </body>
</html>
```



- The browser does not show the tags but uses them them to decide how to display the content.

HTML: HyperText Markup Language

- HTML describes the **content** on a page
- Example `index.html`

Open tag

Attribute

```
<!DOCTYPE html>
<html lang="en">
  <body>
    Hello world!
  </body>
</html>
```

Close tag

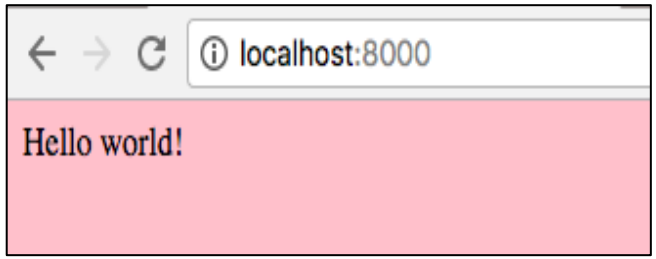
- The browser does not show the tags but uses them them to decide how to display the content.

CSS: Cascading Style Sheets

- CSS describes the **layout** or **style** of a page.

- Example: `style.css`

```
body {
    background: pink;
}
```

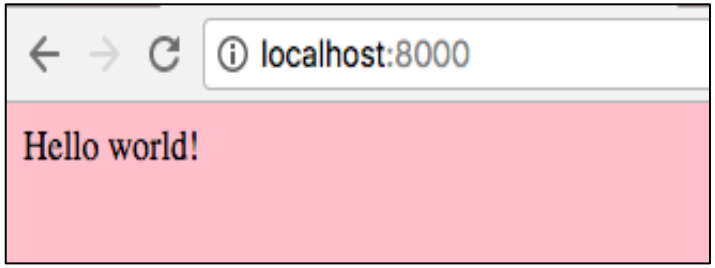


CSS : Cascading Style Sheets

- CSS describes the **layout** or **style** of a page.

- Example `style.css`

```
body {
    background: pink;
}
```



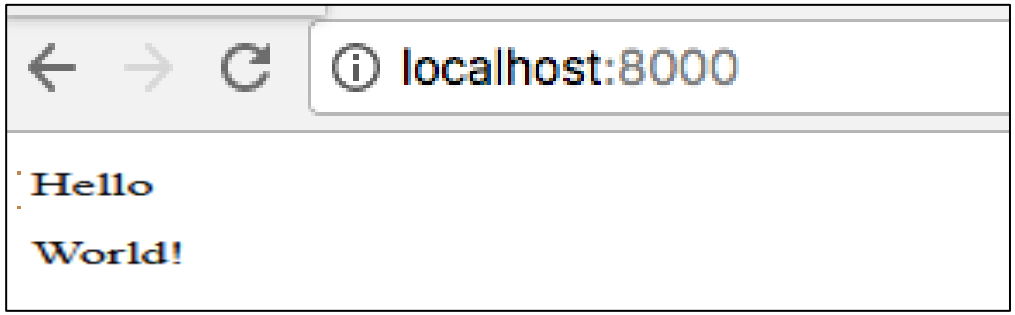
- **Link** to CSS in HTML

```
<html lang="en">
  <head>
    <link rel="stylesheet" type="text/css" href="/style.css">
  </head>
  <body>
    Hello world!
  </body>
</html>
```

HTML: Example

- How can I color each <p> element a different color in `index.html`?

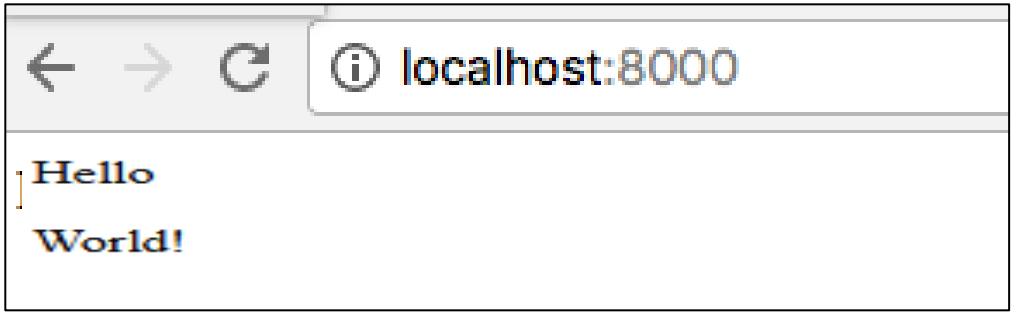
```
<html lang="en">
  <body>
    <P>Hello</P>

    <P>World!</P>
  </body>
</html>
```



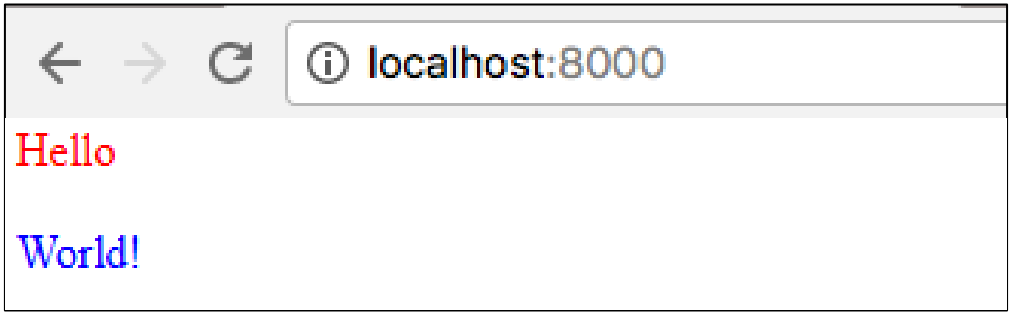
HTML: Example

- How can I color each <p> element a different color in `index.html`?

```
<html lang="en">
  <body>
    <P class="first">Hello</P>
    <P class="second">World! </P>
  </body>
</html>
```



CSS: Example




```
style.css
    .first {
        color: red;
    }
    .second {
        color: blue;
    }
```

```
<html lang="en">
  <head>
    <link rel="stylesheet" type="text/css" href="/style.css">
  </head>
  <body>
    <P class="first">Hello</P>
    <P class="second">world!</P>
  </body>
</html>
```

Static pages

- A *static page doesn't change*
  - Often HTML,CSS, Images files
  - Every user sees the same page

•Example: Server responds with the same files for each request.

```
<html lang="en">
  <body>
    Hello world!
  </body>
</html>
```

```
body {
    background: pink;
}
```

Static page server

- A static file server sends files that don't change
  - HTML, CSS, images, etc.
- Server loads the file from disk and sends to client

- You'll use a static file server in project 1

```
<html lang="en">
  <body>
    Hello world!
  </body>
</html>
```

```
body {
    background: pink;
}
```

How a static page is served

- Client requests a page
- Static file server find the files
- Server sends the file back
- Client’s browser displays the page

```
<html lang="en">
  <body>
    Hello world!
  </body>
</html>
```

```
body {
    background: pink;
}
```

Exercise Overview

- We will
  - Create a static page
  - Run a static file server
  - View the page in a client’s browser

Step 1: Create a static page

- Create **mysite** folder for your website
  - This folder will contain your static files.
- Inside the folder, create a file that is called **index.html**

```
<html lang="en">
  <body>
    Hello from EECS 485!
  </body>
</html>
```

Client

internet

Server

Step 2: Start a static file server

- Start the server from inside the folder

```
> python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 ...
```

- Python is now acting as a static file server

Client

internet

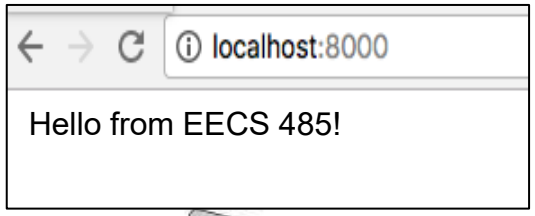
```
<html lang="en">
  <body>
    Hello from EECS 485!
  </body>
</html>
```

```
> python3 -m http.server
```

Server

Step 3: Visit the page

- Open your browser and go to:
  - http://localhost:8000 or
  - http://0.0.0.0:8000



Client

internet

```
<html lang="en">
  <body>
    Hello from EECS 485!
  </body>
</html>
```

Server

```
> python3 -m http.server
```

Agenda

- What is EECS 485 about?
- Logistics
- How the web works
- **Python**

Why Python

- Easy to read and write
- Great for scripting and servers
- Python will be our main backend (server) language

Running Python Code

- Create a file called **add.py**

```
x = 0
x = x + 1
print(x)
```

- Run it in the terminal

```
$ python3 add.py
1
```

Python Interactive Mode

- You can also run Python interactively

```
$ python3
> x = 0
> x = x + 1
> x
1
```

Python Variables

- No type declarations
- Types are inferred automatically

- Example:

```
> x = 10
> Y = 3
> Print (x+y)
13
```

Python Blocks

- Python uses indentation
- Bolacks are defined by whitespace
- No { } like C/C++

- Example:

```
> If x > 10:
>     print(“x is large”)
```

Python Lists

- A list store multiple values

```
>numbers = [10,20,30]
```

- Loop through a list

```
>for n in numbers:
>   print(n)
```

Python Dictionaries

- Dictionaries store key-value pairs:

```
student = { ”name": ”john", ”umid": ”123" }
```

- Access values:

```
>print(student[”name"])
```

Python functions

- Functions group reusable code

```
>def increment(x):
>     return x + 1

>print(increment(5))

6
```

Python standard library

- Python ships with a huge standard library (Built-in tools)
  - https://docs.python.org/3/tutorial/stdlib.html

- Python includes many useful libraries:
  - `pathlib`: filename manipulations
  - `sys`: system functions, e.g., `exit`
  - `json`: read/write JSON files
  - `shutil`: file manipulations

- You will use these in Project 1.

Python third party libraries

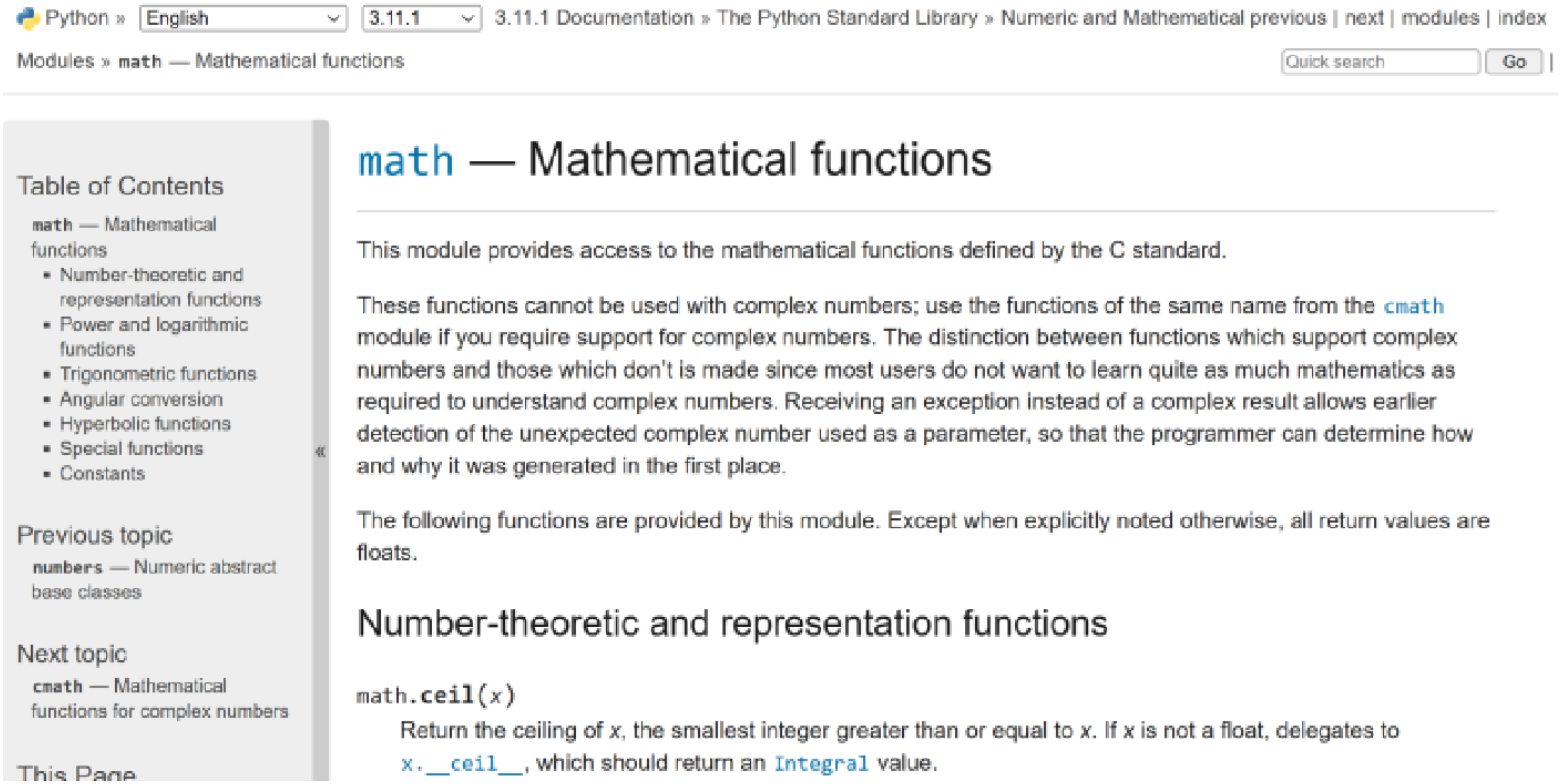
- Some libraries are not built in, but you will need them for project 1
  - `jinja2`: HTML template
  - `click`: command-line tools and argument parsing

- Install with

```
$ pip install jinja2 click
```

Best Python documentation

- Python.org



Python » English 3.11.1 3.11.1 Documentation » The Python Standard Library » Numeric and Mathematical previous | next | modules | index

Modules » math — Mathematical functions

Quick search Go |

Table of Contents

math — Mathematical functions
- Number-theoretic and representation functions
- Power and logarithmic functions
- Trigonometric functions
- Angular conversion
- Hyperbolic functions
- Special functions
- Constants

Previous topic

numbers — Numeric abstract base classes

Next topic

cmath — Mathematical functions for complex numbers

This Page

math — Mathematical functions

This module provides access to the mathematical functions defined by the C standard.

These functions cannot be used with complex numbers; use the functions of the same name from the cmath module if you require support for complex numbers. The distinction between functions which support complex numbers and those which don't is made since most users do not want to learn quite as much mathematics as required to understand complex numbers. Receiving an exception instead of a complex result allows earlier detection of the unexpected complex number used as a parameter, so that the programmer can determine how and why it was generated in the first place.

The following functions are provided by this module. Except when explicitly noted otherwise, all return values are floats.

Number-theoretic and representation functions

math.ceil(x)

Return the ceiling of x, the smallest integer greater than or equal to x. If x is not a float, delegates to x.__ceil__, which should return an Integral value.

Summary

- Please read syllabus and eecs485.org
- *How the web works (Request response cycle):* client requests and server responds
- *Static page* is the same every time the page is loaded
- *Static file server* sends a copy of a file from disk
- Python programs are run by an *interpreter*

Your to-do list

- Learn HTML
  - https://www.w3schools.com/html/html_intro.asp
- Learn (a little) CSS
  - https://www.w3schools.com/html/html_css.asp
- Learn Python
  - https://docs.python.org/3/tutorial/index.html
- Get started on Project 1
  - Link on eecs485.org

Waitlist

- Wolverine Access handles the waitlist automatically
- When one enrolled student drops, Wolverine Access issues permission via email to one waitlisted student
  - Follow the instructions in the email
- Unfortunately, it's hard to predict how many students will drop